Master Engineering Systems

Major Project Report: Progress

Developing a Scalable Computer-Vision Pipeline for Human–Interaction Element Detection in Hospital Environments

Pavel Petrovski

HAN University of Applied Sciences
Supervisors: Siddharth Ajaykumar, Jan Benders, Ben Pyman

March 16, 2026

Master Engineering Systems – Major Project
Cyber-Physical Systems
HAN University of Applied Sciences
Arnhem

Preface

This report presents the work carried out as part of the Master thesis project in Cyber-Physical Systems at HAN University of Applied Sciences. The project was conducted in collaboration with Ambyon and HAN Automotive Research.

The objective of the thesis is to design and evaluate a scalable computer-vision pipeline for interaction-point perception in hospital environments. All work described in this report was performed by the author.

Disclaimer

This Project Report was written independently by the author as part of the Master of Engineering Systems program at HAN University of Applied Sciences.

During the preparation of this report, the generative artificial intelligence tool *ChatGPT* (OpenAI, 2025) was used selectively to support the writing process. The tool was employed to assist with improving sentence structure, refining academic language, and suggesting minor text rephrasing.

All technical content, system design decisions, experimental implementation, training procedures, evaluation methods, and results presented in this report are the original work of the author. The use of the AI tool did not replace independent analysis, interpretation of scientific literature, or critical reasoning.

The use of AI complies with the *MES Policy on the Use of Generative AI*, ensuring informed, transparent, ethical, and responsible application.

Summary

This project focuses on the design and evaluation of a computer-vision pipeline for detecting and interpreting interaction points, such as elevator buttons, in hospital environments. The work is motivated by the need for robust perception capabilities in service robots operating across different hospital infrastructures.

A scalable workflow is proposed that combines instance segmentation, semi-automatic annotation, and iterative model retraining. A custom dataset is created by extending existing data with instance-level annotations using a CVAT-based annotation pipeline. Multiple instance segmentation models are trained using transfer learning, evaluated using standard and task-oriented metrics, and prepared for deployment on embedded hardware.

The project demonstrates a complete perception pipeline, from dataset creation to embedded deployment, and provides a foundation for future extensions toward fully autonomous robot interaction in clinical environments.

Acronyms and Abbreviations

AI Artificial Intelligence.

AP Average Precision.

APA American Psychological Association.

CNN Convolutional Neural Network.

COCO Common Objects in Context.

CPU Central Processing Unit.

CV Computer Vision.

CVAT Computer Vision Annotation Tool.

DL Deep Learning.

FP16 Half-precision floating point format.

FPS Frames Per Second.

GPU Graphics Processing Unit.

IEEE Institute of Electrical and Electronics Engineers.

IoU Intersection over Union.

mAP Mean Average Precision.

Mask R-CNN Mask Region-Based Convolutional Neural Network.

ML Machine Learning.

NVIDIA Jetson Embedded GPU platform for edge AI.

OCR Optical Character Recognition.

ONNX Open Neural Network Exchange.

RGB Red–Green–Blue.

RGB-D Red–Green–Blue plus Depth.

RNN Recurrent Neural Network.

ROS Robot Operating System.

ROS 2 Robot Operating System 2.

SLAM Simultaneous Localization and Mapping.

SOTA State of the Art.

TBD To Be Determined.

TensorRT TensorRT.

WSL2 Windows Subsystem for Linux 2.

XML Extensible Markup Language.

YOLO You Only Look Once.

Contents

1 Introduction

This introduction includes the background information, problem definition, project objective and the research questions.

1.1 Background

Robotics in healthcare is moving from experimental pilots to practical deployment, supporting tasks such as logistics, delivery of medical supplies, and environmental assistance. These systems are intended not to replace clinical staff, but to support them by reducing repetitive physical work and enabling more time for patient care [6, 14]. As hospitals face increasing operational pressure and staff shortages, autonomous robots capable of safe navigation and interaction with the environment hold significant potential.

Hospitals are typically multi-floor environments that rely on infrastructure such as elevators, automatic doors, and access-control systems. For a robot to move freely and perform tasks across different floors, it must be able to detect and interact with these human–machine interfaces, including elevator buttons and door panels. Reliable perception of such interaction elements is therefore a key capability for achieving truly autonomous navigation in hospital settings. Without this, robots remain confined to limited single-floor operation or require human assistance for vertical transport.

Figure 1.1: CareBot prototype at HAN Automotive Research

At *HAN Automotive Research*, the CareBot platform is being developed as an educational and research project that allows students and researchers to experiment with robotic perception, planning, and control in realistic environments. While not designed to replicate the Ambyon ONE robot, the CareBot in this project will be equipped with a similar sensor stack and processing unit, enabling realistic simulation of Ambyon's operational conditions. The Ambyon ONE itself is a service robot currently under development by *Ambyon*, a Dutch startup focused on robotics for healthcare and logistics applications.

The CareBot system uses an Red–Green–Blue plus Depth (RGB-D) camera and Embedded GPU platform for edge AI (NVIDIA Jetson) computing module to enable on-device perception suitable for real-time operation in clinical environments. A key research challenge is enabling the robot to accurately detect and classify interaction points such as elevator buttons, access readers, and door panels under varying lighting, reflections, occlusions, and diverse panel designs - while maintaining efficient performance within embedded hardware constraints.

This project contributes to that research direction by designing and evaluating a scalable

computer-vision workflow for interaction-point detection on embedded systems. The work includes dataset creation, model design and optimisation, evaluation on prototype hardware, and preparation for future extensions. The resulting system will serve as a foundation for autonomous hospital mobility and support the development of assistive robotic technologies.

1.2 Problem Definition

The Ambyon ONE robot currently lacks a reliable computer-vision system to detect and recognize human-interaction elements (e.g., elevator and door control interfaces) in hospital environments, preventing autonomous navigation and interaction with building infrastructure.

1.3 Project Objective

Develop and validate an efficient and scalable computer-vision workflow that enables a service-robot platform to autonomously detect and classify human-interaction elements in hospital environments using embedded hardware.

1.4 Research Question(s)

Main Research Question:
How can an efficient and scalable computer-vision workflow be designed and implemented to enable a service robot to detect and interpret human-interaction elements in hospital environments using embedded hardware?

Sub-questions:

1. What are the relevant human–interaction points in hospital environments that should be detected and classified by the computer-vision workflow?
2. What computer-vision methods and model architectures are most suitable for detecting and classifying human-interaction elements in service-robot environments?
3. How can a high-quality dataset be designed and constructed for achieving robust model performance?
4. Which model-optimization techniques support real-time inference on embedded hardware while preserving detection accuracy?
5. How can the workflow support scalable and efficient model improvements, including potential data collection and retraining during deployment?

1.5 Outline of the report

Chapter 1 introduces the background, motivation, and objectives of the project. Chapter 2 reviews relevant literature on interaction-point perception, instance segmentation, and embedded computer-vision systems. Chapter 3 describes the methodology, including dataset preparation,

annotation strategy, model training, evaluation, and deployment considerations. Chapter 4 presents the experimental results and quantitative evaluation. Chapter 5 discusses the findings and their implications, and Chapter 6 concludes the report and outlines future work.

2 Literature Review

This chapter reviews the relevant scientific literature used in the Project, related to interaction-point perception in hospital environments, with a focus on instance segmentation, dataset design, annotation strategies, training and deployment of computer-vision models on embedded systems.

2.1 Perception of human–machine interaction elements

Human–machine interaction elements such as elevator buttons, door panels, and access readers are important targets for indoor service robots operating in hospital environments. Previous work in healthcare and service robotics shows that reliable interaction with such infrastructure is required for autonomous operation across multiple floors and restricted areas [6, 14].

From a computer-vision perspective, these interaction elements are challenging to detect because they are small, visually similar, and often arranged closely together on control panels. Studies on visual perception for robotic interaction report that elevator panels typically contain many buttons with minimal visual differences, which makes robust localization difficult under varying lighting conditions, reflections, and viewing angles [2].

General indoor-robot perception systems often focus on detecting large objects or structural elements and assume that interaction with building infrastructure is externally supported or environment-specific. However, research on autonomous service robots emphasizes that scalable deployment requires perception systems that can directly identify actionable interface elements from camera and sensor data, without relying on prior knowledge of the environment [21].

These findings indicate that interaction-point perception is a fine-grained vision problem in which individual interface elements must be localized accurately within visually dense scenes. This observation motivates the need for vision methods that go beyond coarse detection of panels or regions and instead support precise identification of individual interaction elements.

2.2 Formulating interaction-point perception as a vision problem

Vision-based perception problems in robotics are commonly formulated as image classification, object detection, or semantic segmentation tasks. Each formulation implies different assumptions about the structure of the scene and the required output, which directly affects suitability for robotic interaction.

Image classification assigns a single label to an entire image and is therefore unsuitable for scenarios where multiple interaction elements appear simultaneously. Object detection extends this formulation by predicting bounding boxes and class labels for individual objects, in the case of the project those are elevator buttons. While detection-based approaches have been widely used for indoor perception tasks, bounding boxes provide only coarse spatial localization and do not capture the precise shape of interaction elements [17, 16].

Semantic segmentation addresses this limitation by assigning a class label to each pixel in the

image. Formally, semantic segmentation can be expressed as a pixel-wise classification function

$$f_\theta : \mathbb{R}^{H\times W\times C} \to \{1,\dots,K\}^{H\times W},$$

where H and W denote image height and width, C the number of input channels, and K the number of semantic classes [12]. Although this formulation enables precise localization, all pixels belonging to the same class are treated as a single region. As a result, semantic segmentation cannot distinguish between multiple instances of the same object class.

For interaction-point perception, this limitation is critical. Elevator panels often contain many visually similar buttons, each corresponding to a distinct action. Semantic segmentation assigns the same class label to all pixels belonging to a given category and therefore cannot distinguish between multiple instances of the same object class [12, 5]. As illustrated in Figure 2.1, a semantic segmentation output would label all button pixels identically, without separating individual buttons. This lack of instance-level differentiation makes semantic segmentation unsuitable for downstream decision-making and physical interaction, where each button must be treated as a separate actionable element.

Instance segmentation extends semantic segmentation by jointly performing object detection and pixel-wise classification, producing a separate mask for each object instance. This formulation can be expressed as a set of instance masks

$$\mathcal{M} = \{M_1, M_2, \dots, M_N\},$$

where each $M_i \in \{0,1\}^{H\times W}$ represents a binary mask for one object instance [5]. Instance segmentation therefore enables both precise spatial localization and differentiation between multiple objects of the same class. Figure 2.1 illustrates the conceptual differences between image classifi-

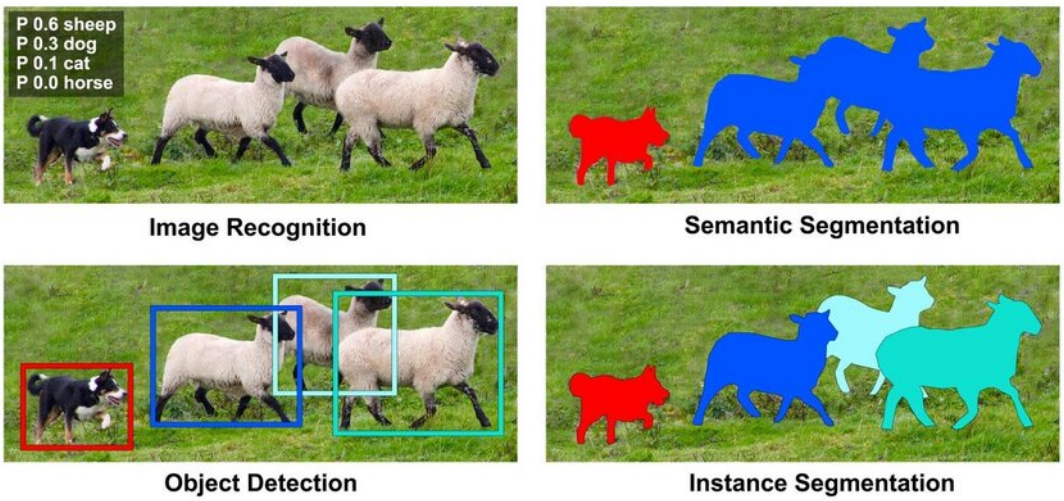


Figure 2.1: Comparison of image classification, object detection, semantic segmentation, and instance segmentation.

cation, object detection, semantic segmentation, and instance segmentation for visual perception tasks.

Based on findings in the literature and the requirements of interaction-point detection, in-

stance segmentation provides the most appropriate problem formulation for this project. It allows individual buttons to be identified as distinct entities while preserving pixel-level accuracy, which is essential for reliable robotic interaction in dense and visually complex control interfaces.

2.3 Public datasets and annotation strategies.

Public datasets for vision-based detection of elevator interfaces are limited, as most indoor robotics datasets focus on navigation or scene understanding rather than interaction elements [19]. This limits their direct applicability to interaction-point perception tasks.

The IRSO 2018 dataset provides Red–Green–Blue (RGB) images of elevator panels captured in realistic indoor environments and was originally introduced for object detection tasks [13]. Due to its focus on real elevator interfaces and visual diversity, it is well suited for studying interaction-point perception in service robotics.

Recent work reports a large-scale dataset with pixel-wise annotations for elevator button segmentation [11]. However, despite being described in the literature, the corresponding semantic segmentation dataset is not publicly available at the time this project is made and therefore cannot be used.

To enable instance-level interaction-point detection, this work reuses images from the IRSO 2018 dataset and replaces the original bounding box annotations with instance segmentation masks. Annotations are stored in the Common Objects in Context (COCO) format, which supports instance-level labeling and is widely used across computer vision frameworks, facilitating reuse and future extensions [10].

2.4 Model design principles, evaluation metrics, and semantic interpretation

Instance segmentation models for robotic perception typically follow a modular design in which feature extraction, object localization, and pixel-level mask prediction are handled by separate components within a single network. This design enables shared visual representations while supporting accurate instance-level localization, which is required for interaction-point perception in dense control interfaces.

Region-based instance segmentation models such as Mask Region-Based Convolutional Neural Network (Mask R-CNN) achieve high localization accuracy by explicitly separating region proposal, classification, and mask prediction stages [5]. Due to this design, Mask R-CNN is widely used as a reference architecture and benchmark model in instance segmentation research. However, the multi-stage nature of the model results in high computational cost and memory usage, which limits its suitability for real-time deployment on embedded robotic platforms.

To address these constraints, more lightweight instance segmentation architectures have been proposed that integrate detection and mask prediction into a single-stage pipeline. You Only Look Once (YOLO)-based segmentation models, such as YOLOv8-Seg, trade a moderate reduction in segmentation accuracy for substantially improved inference speed and computational

efficiency [8]. These models have demonstrated competitive performance in real-time and embedded settings, making them suitable candidates for deployment on resource-constrained robotic systems.

Model performance is commonly evaluated using overlap-based metrics that measure agreement between predicted masks and ground-truth annotations. Intersection over Union (IoU) is defined as

$$\text{IoU} = \frac{|M_p \cap M_{gt}|}{|M_p \cup M_{gt}|},$$

where M_p and M_{gt} denote the predicted and ground-truth masks. Average Precision (Average Precision (AP)), computed over multiple IoU thresholds, is widely used as a standard evaluation metric for instance segmentation tasks [10].

In many perception pipelines for elevator interfaces, instance localization is followed by a semantic interpretation stage that determines the functional meaning of each detected button. This is commonly achieved using Optical Character Recognition (OCR) or dedicated symbol classification models applied to the isolated button regions [11]. Accurate instance segmentation is a critical enabling step for such approaches, as it provides clean, well-aligned inputs for subsequent classification or text recognition.

In this work, instance segmentation is treated as the primary perception task, as reliable instance-level localization is a prerequisite for any semantic interpretation of interaction elements. The resulting pipeline is therefore designed to support integration of OCR or custom button classifiers, enabling the robot to associate detected buttons with their intended functions in downstream processing stages.

2.5 Embedded deployment constraints in robotic perception

Vision-based perception systems deployed on mobile service robots must operate under strict computational and energy constraints. Prior work in robotic vision reports that deep-learning models deployed on embedded platforms are limited by onboard memory, processing power, and power consumption, which directly affects real-time performance [3].

Studies evaluating deep-learning inference on NVIDIA Jetson-class hardware show that models achieving high accuracy on desktop Graphics Processing Units (GPUs) often exhibit increased latency and memory usage when deployed on embedded devices [9]. These findings indicate that benchmark accuracy alone is not a sufficient criterion for model selection in robotic perception systems.

For instance segmentation tasks, this trade-off is particularly relevant, as mask prediction introduces additional computational overhead compared to bounding-box detection. Research on efficient model design highlights that architectural choices such as backbone depth and feature reuse have a significant impact on inference efficiency [20]. Based on these insights, this project treats model efficiency and deployability as primary design constraints alongside segmentation accuracy.

Consequently, instance segmentation models considered in this work are evaluated not only with respect to localization performance, but also in terms of their suitability for embedded deployment. This literature-driven perspective directly informs the model selection and opti-

mization strategies described in Chapter 3(Methodology).

2.6 Scalability, robustness, and reproducibility.

Several studies emphasize that robust perception systems require more than a single well-performing model. Instead, scalability is achieved through structured data management, consistent annotation practices, and reproducible training pipelines that support incremental dataset expansion and model updates [4]. These practices reduce sensitivity to domain-specific bias and enable adaptation to new environments without redesigning the entire system.

From these findings, this project adopts a pipeline-oriented approach in which dataset creation, annotation, training, and evaluation are treated as modular and repeatable processes. Rather than optimizing a single model for a fixed dataset, the focus is placed on maintaining a perception workflow that can be extended with new data and retrained as deployment conditions evolve.

2.7 Summary and identified research gap

The reviewed literature shows that reliable perception of interaction elements in robotic systems requires instance-level localization, suitable annotation strategies, and awareness of embedded deployment constraints. Existing studies demonstrate the technical feasibility of these components, but often address them in isolation.

At the same time, publicly available datasets for elevator interfaces are limited, and existing annotations are commonly designed for object detection rather than segmentation tasks. In addition, many approaches focus on model performance without explicitly considering scalability, reproducibility, or deployment on embedded robotic platforms.

This project addresses these gaps by repurposing a realistic elevator dataset with instance-level annotations and by developing a scalable and deployment-aware computer-vision pipeline for interaction-point perception in hospital environments. The following chapter describes the methodology used to realize this approach.

3 Methodology

This chapter describes the methodology used to develop a computer-vision pipeline for interaction-point perception in hospital environments. The project is conducted with the objective of building a scalable and deployment-oriented perception system that can be iteratively improved across different hospital settings and reused within a robot fleet.

3.1 Overview of the proposed perception pipeline

The proposed methodology focuses on instance-level localization of interaction elements such as elevator buttons and control panels. Instance segmentation is used to detect individual button instances at pixel level, allowing precise localization and separation of closely spaced elements. This capability provides the basis for subsequent interpretation of button functions and autonomous interaction.

The methodology is designed to support an iterative training and deployment workflow. An initial base model is deployed on the robot, where it performs interaction-point perception and collects visual data during operation. Selected data can be transferred to an offboard system, where annotations may be reviewed, corrected, or extended before retraining the perception models. The updated models are then redeployed to the robot and become the new default for continued operation.

This workflow enables gradual adaptation to new hospital environments and allows knowledge acquired in one deployment to be reused and refined in subsequent deployments. The target end-to-end training and deployment process toward which this project is developed is illustrated in Figure 3.1.

The following sections describe each stage of the methodology in detail, beginning with dataset selection and preparation.

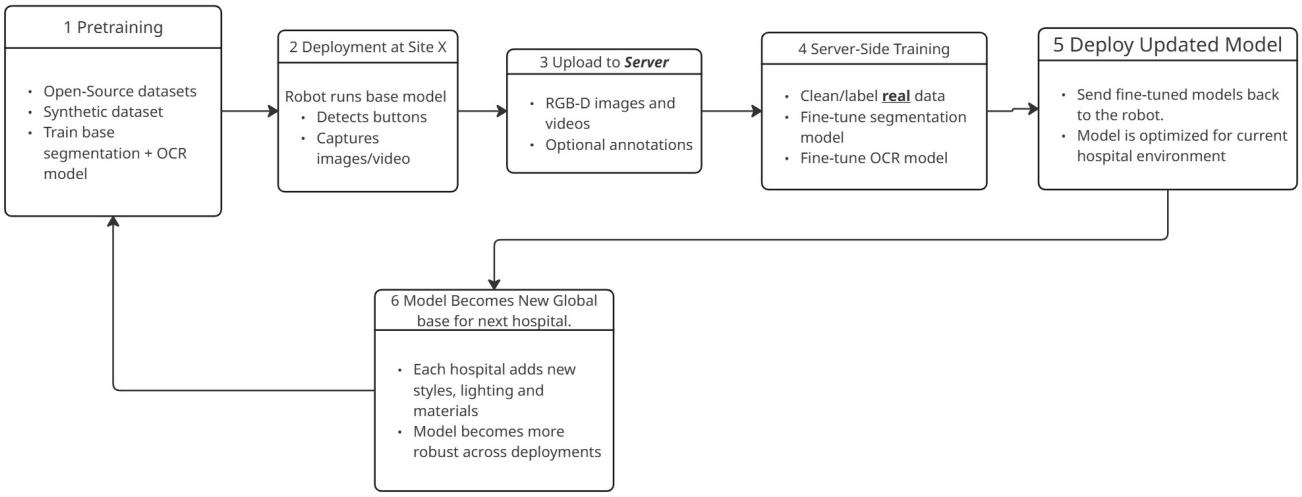


Figure 3.1: Target workflow for scalable training and deployment of interaction-point perception models across multiple hospital environments.

3.2 Dataset selection and preparation

This project uses a dataset derived from the IRSO2018 collection, which contains images of elevator panels and buttons captured in realistic indoor environments. This section corresponds to the initial model-development step in Figure 3.1, detailing the dataset and its preparation before the first training cycle. The origin and relevance of this dataset were discussed in Literature Review; therefore, this section focuses on its structure and preparation for use in this project.

The dataset consists of a total of 2000 images. Of these, 1400 images are assigned to the training set (`train_set`) and 600 images to the test set (`test_set`). Each subset follows the same directory structure, containing an `images` folder with the raw RGB images and an `annotations` folder with the corresponding annotation files.

In the original dataset, annotations were provided as bounding boxes stored in Extensible Markup Language (XML) format. For the purposes of this project, the directory structure was preserved, but the annotation format was replaced with COCO-style JSON files to support instance-level segmentation. This ensures compatibility with modern instance segmentation frameworks while maintaining a clear separation between training and test data.

Figure 3.2: Example raw image from the dataset.

The resulting dataset structure provides a consistent and reusable foundation for annotation, model training, and evaluation, as described in the following sections.

3.3 Annotation strategy and workflow

Instance-level ground truth was created using a semi-automatic workflow that combines model-based pre-annotation with manual verification. This strategy was used to efficiently generate high-quality instance segmentation annotations while maintaining consistency across the dataset.

Annotations were created in Computer Vision Annotation Tool (CVAT) [18], deployed locally using Docker [15] within a Windows Subsystem for Linux 2 (WSL2)-based Linux environment on a Windows host. Automatic annotation was enabled through CVAT–Nuclio integration, where the model was deployed as a Nuclio serverless function [7]. The instance segmentation model used for automatic annotation was Mask R-CNN [5].

During automatic annotation, CVAT sends task images to the Nuclio function, where Mask R-CNN predicts binary masks for each detected button instance. These masks are converted

into polygon shapes within the Nuclio function and returned to CVAT as annotation data. The overall workflow used in this project is shown in Figure 3.3.

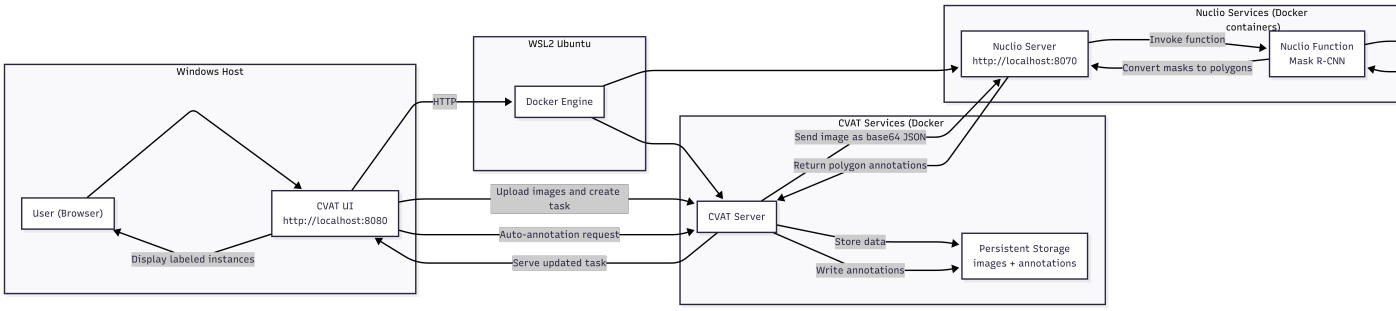


Figure 3.3: Semi-automatic annotation workflow on a Windows host using WSL2 and Docker. CVAT sends images to a Nuclio function running Mask R-CNN, which returns polygon instance annotations derived from predicted binary masks.

To support iterative improvement, the dataset was annotated in batches of approximately 150 images. After each batch, the model was retrained using the newly annotated data and reused for subsequent annotation rounds. Model checkpoints were created after approximately 100, 450, and 900 annotated images, enabling progressively improved pre-annotations and reduced manual correction effort.

All automatically generated annotations were also manually reviewed and corrected when needed to ensure accurate instance boundaries, correct separation of adjacent buttons, and consistent labeling across both the training and test sets.

Figure 3.4: Example of an annotated image showing instance segmentation masks.

3.4 Dataset quality assessment.

Note: This section is not done, and the text within it will most probably be changed drastically. However, I intent to keep structure and general idea the same. One subsection for model assisted image quality assessment, and one for statistical image quality metrics.

A systematic dataset quality assessment was performed to verify whether image-level degradation could negatively affect model performance.

3.4.1 Statistical quality metrics.

For every image, the following metrics were computed:

- **Sharpness**, estimated using the variance of the Laplacian:

$$\sigma^2_{\text{lap}} = \text{Var}(\nabla^2 I) \tag{3.1}$$

  where I is the grayscale image.

- **Brightness**, defined as the mean grayscale intensity.

- **Edge density**, defined as the fraction of pixels with gradient magnitude above the 90th percentile.

—

3.4.2 Model-assisted image assessment.

In addition to image-level quality analysis, a model-assisted approach was used to identify potentially harmful training samples.

Method. A trained instance segmentation model was used to evaluate each training image individually. For each image, the following metrics were computed:

- Mean Intersection over Union:

$$\text{IoU} = \frac{|M_{\text{pred}} \cap M_{\text{gt}}|}{|M_{\text{pred}} \cup M_{\text{gt}}|} \tag{3.2}$$

- Mean detection confidence:

$$\bar{c} = \frac{1}{N} \sum_{i=1}^{N} p_i \tag{3.3}$$

  where p_i is the softmax score of the i-th predicted instance.

- Number of false positives and false negatives.

Images were ranked using a composite criterion favoring low IoU, high false positives, and high confidence incorrect predictions. Based on those criteria, a harm-score is measured, based on which training set images are filtered and the potentially harmful ones are selected for manual inspection.

Inspection. Manual inspection of the lowest-ranked samples revealed two categories:

- Hard but valid images (extreme viewpoints, zoom levels, or geometric distortions).

- Negative images without buttons, occasionally producing false positives.

After retraining the model with excluded possibly "harmful" samples, it was observed that removing hard samples reduced recall and segmentation accuracy, while removing a small number of negative samples slightly improved precision.

3.5 Model architectures and training strategy

Two instance segmentation model families are employed in this project, each serving a distinct role within the overall perception pipeline. Mask R-CNN is used as a high-accuracy reference model and as the backbone of the semi-automatic annotation workflow (Section 3.3), while YOLOv8-Seg is trained and evaluated as a deployment-oriented solution suitable for embedded hardware.

Common training strategy

All models are trained using a transfer learning approach. Networks are initialized from pre-trained checkpoints and fine-tuned on the project-specific dataset, rather than being trained from scratch. This strategy improves convergence speed and generalization performance, particularly given the moderate dataset size.

The dataset is split into training and validation subsets using an 80/20 ratio with a fixed random seed (42) to ensure reproducibility. Standard data augmentation techniques are applied during training to improve robustness to viewpoint and lighting variations. These include random horizontal flipping with probability $p = 0.5$ and color jitter with brightness, contrast, and saturation factors set to 0.1.

Mask R-CNN training

Mask R-CNN is trained using a custom PyTorch-based pipeline with COCO-formatted instance annotations. Training is performed for 50 epochs with a batch size of 2, reflecting the higher memory requirements of region-based instance segmentation models.

Optimization is performed using the AdamW optimizer with a learning rate of 5×10^{-4} and weight decay of 1×10^{-4}. A ReduceLROnPlateau learning rate scheduler is employed, reducing the learning rate by a factor of 0.5 when the validation loss does not improve for three consecutive epochs. Early stopping is enabled with a patience of 10 epochs based on validation loss.

Mask R-CNN is retrained iteratively as the annotated dataset grows. This iterative training is applied exclusively to the Mask R-CNN model, as it is used to generate pre-annotations during the semi-automatic annotation process. Model checkpoints are created after approximately 100, 450, and 900 annotated training images, enabling progressively improved annotation quality and reduced manual correction effort.

YOLOv8-Seg training

YOLOv8-Seg is trained using the Ultralytics framework with the pretrained `yolov8s-seg.pt` checkpoint. Training is performed for 50 epochs with a batch size of 8 and an input resolution of 640×640 pixels. The default Ultralytics hyperparameters are used for the training. Early stopping patience is set to 10 epochs based on validation loss. 3 epochs for learning rate warmup, gradually increasing the learning rate from a low value to the initial learning rate to stabilize training early on

Unlike Mask R-CNN, YOLOv8-Seg is not retrained iteratively during annotation. Instead, it is trained on the finalized annotated dataset and evaluated as a candidate model for real-time deployment on embedded hardware.

Loss functions and optimization objectives

Both instance segmentation models are trained by minimizing a composite loss function that jointly optimizes object classification, localization, and mask prediction. Although Mask R-CNN and YOLOv8-Seg differ architecturally, their optimization objectives are conceptually aligned.

Mask R-CNN. The training objective of Mask R-CNN is defined as a multi-task loss composed of three terms [5]:

$$\mathcal{L}_{\text{Mask R-CNN}} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{box}} + \mathcal{L}_{\text{mask}}, \tag{3.4}$$

where $\mathcal{L}_{\text{cls}}$ denotes the classification loss, implemented as categorical cross-entropy; $\mathcal{L}_{\text{box}}$ represents bounding-box regression using the smooth L_1 loss; and $\mathcal{L}_{\text{mask}}$ is a pixel-wise binary cross-entropy loss applied to each predicted instance mask.

YOLOv8-Seg. YOLOv8-Seg integrates detection and segmentation into a single-stage architecture. The total training loss is expressed as a weighted sum of classification, localization, and segmentation losses [8]:

$$\mathcal{L}_{\text{YOLO}} = \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{seg}}\mathcal{L}_{\text{seg}}, \tag{3.5}$$

where $\mathcal{L}_{\text{seg}}$ represents the segmentation loss applied to the predicted mask outputs, and the weighting coefficients λ control the relative contribution of each task during optimization.

Despite architectural differences, both models optimize comparable objectives, enabling consistent evaluation of segmentation accuracy and computational efficiency across reference and deployment-oriented architectures.

OCR-based button interpretation

Following instance segmentation, the detected button regions are passed to a semantic interpretation stage. Segmented button instances are cropped and processed by an OCR-based recognition module to extract character labels or symbols associated with each button. This stage enables the robot to associate detected interaction points with their functional meaning and supports downstream decision-making during autonomous operation.

3.6 Evaluation metrics and methodology

The performance of the instance segmentation models is evaluated using a combination of standard COCO metrics and task-oriented instance-level metrics. This dual evaluation strategy allows both standardized benchmarking and direct assessment of model suitability for robotic interaction tasks.

3.6.1 Intersection over Union

Segmentation quality is measured using Intersection over Union (IoU). For a predicted instance mask M_p and the corresponding ground-truth mask M_{gt}, IoU is defined as:

$$\text{IoU} = \frac{|M_p \cap M_{gt}|}{|M_p \cup M_{gt}|} \tag{3.6}$$

IoU values range from 0 (no overlap) to 1 (perfect overlap). In this project, IoU is computed at the instance level and used both for matching predicted and ground-truth instances and for computing summary statistics.

3.6.2 Precision, recall, and F1-score

Instance-level detection performance is quantified using precision, recall, and F1-score. A predicted instance is considered a true positive if its IoU with an unmatched ground-truth instance exceeds a threshold of 0.5. Precision and recall are defined as:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3.7}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3.8}$$

where TP, FP, and FN denote the number of true positives, false positives, and false negatives, respectively.

The F1-score, which balances precision and recall, is computed as:

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.9}$$

These metrics provide intuitive insight into detection reliability and are particularly relevant for robotic interaction tasks, where both missed detections and false positives can lead to failed interactions.

3.6.3 Average Precision under the COCO protocol

To enable standardized benchmarking, model performance is additionally evaluated using the COCO instance segmentation protocol. Average Precision (AP) is computed as the area under the precision–recall curve:

$$\text{AP} = \int_0^1 p(r)\, dr \tag{3.10}$$

where $p(r)$ denotes precision as a function of recall r. Following the COCO evaluation methodology, AP is reported as the mean over multiple IoU thresholds ranging from 0.50 to 0.95 in increments of 0.05. Additional metrics such as AP_{50} and AP_{75} are also reported to provide insight into performance at specific overlap thresholds.

3.6.4 Mean IoU and runtime performance

For correctly matched instances, the mean IoU is computed across the test set to assess overall segmentation accuracy. In addition to accuracy metrics, runtime performance is measured by recording per-image inference time. Average inference time and frames per second (Frames Per Second (FPS)) are reported to evaluate suitability for real-time deployment on embedded hardware.

3.6.5 Evaluation protocol

All evaluations are conducted on a held-out test set that is not used during training or validation. Identical datasets, matching criteria, and thresholds are used across all model architectures to ensure fair comparison. COCO-based metrics are computed using the official evaluation tools, while precision, recall, F1-score, mean IoU, and runtime statistics are computed using a custom evaluation pipeline.

The evaluation methodology described in this section forms the basis for the quantitative comparison of models presented in Chapter 4.

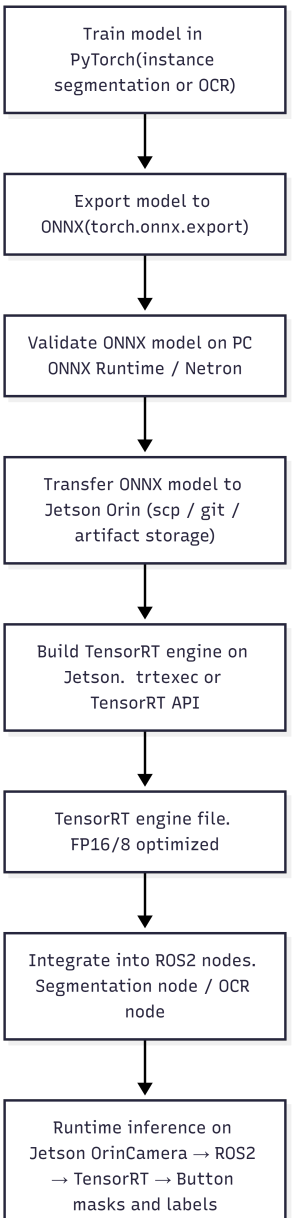
3.7 Embedded deployment and optimization

To enable real-time execution on embedded hardware, and wrap up step 1 of the workflow strategy in Figure 3.1, trained models are optimized and deployed on a NVIDIA Jetson Orin platform. The deployment workflow is designed to be identical for both the instance segmentation model and the OCR-based button classification model.

Models are first trained in PyTorch and exported to the Open Neural Network Exchange (ONNX) [1] format. The exported ONNX models are validated on a desktop system using ONNX Runtime and model inspection tools to ensure numerical correctness and structural compatibility prior to deployment.

Validated ONNX models are then transferred to the NVIDIA Jetson Orin device, where TensorRT is used to build optimized inference engines. Engine generation is performed directly on the target device to ensure compatibility with the specific GPU architecture. Half-precision floating point format (FP16) precision is used to reduce memory footprint and improve inference speed while maintaining sufficient numerical accuracy.

The resulting TensorRT engines are integrated into Robot Operating System 2 (ROS 2) nodes re-



sponsible for perception tasks. At runtime, image data from the RGB-D camera is passed through the ROS 2 pipeline and processed by the TensorRT engines to produce instance segmentation masks and corresponding button labels.

The complete deployment and optimization workflow is illustrated in Figure 3.5. This approach ensures a consistent and efficient deployment pipeline and enables fair evaluation of different model architectures under identical runtime constraints.

Bibliography

[1] Bai, J., Lu, F., Zhang, K., et al. (2019). Onnx: Open neural network exchange. In *Proceedings of the 2019 ACM International Conference on Multimedia Systems*, pages 460–466.

[2] Bernard, A., Hörnle, T., and Dillmann, R. (2020). Vision-based perception of control interfaces for robotic manipulation. *Robotics and Autonomous Systems*, 124:103386.

[3] Chen, T., Chen, Z., and Chen, Y. (2018). Deep learning in robotics: A review of recent research. *Advanced Robotics*, 32(5):201–214.

[4] Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., Franke, U., Roth, S., and Schiele, B. (2016). The cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3213–3223.

[5] He, K., Gkioxari, G., Dollár, P., and Girshick, R. (2017). Mask r-cnn. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2961–2969.

[6] Holland, J. et al. (2021). Service robots in the healthcare sector. *Robotics*, 10(1):47.

[7] Iguazio (2020). Nuclio: High-performance serverless event and data processing platform. `https://nuclio.io`.

[8] Jocher, G., Chaurasia, A., and Qiu, J. (2023). Yolov8: Ultralytics next-generation yolo models. *arXiv preprint arXiv:2305.09972*.

[9] Kang, Y., Kim, H., and Lee, J. (2020). Edge ai: On-device intelligence for embedded systems. *IEEE Consumer Electronics Magazine*, 9(4):20–27.

[10] Lin, T.-Y., Maire, M., Belongie, S., et al. (2014). Microsoft coco: Common objects in context. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 740–755.

[11] Liu, J., Fang, Y., Zhu, D., Ma, N., Pan, J., and Meng, M. Q.-H. (2021). A large-scale dataset for benchmarking elevator button segmentation and character recognition. *IEEE Transactions on Industrial Informatics*.

[12] Long, J., Shelhamer, E., and Darrell, T. (2015). Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3431–3440.

[13] Martínez, M., Montero, R., and Pérez, E. (2018). Irso: A dataset for interactive robotic service operations in indoor environments. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1234–1241.

[14] Masala, G. L. et al. (2025). Artificial intelligence and assistive robotics in healthcare: A narrative review. *International Journal of Environmental Research and Public Health*, 22(5):781.

[15] Merkel, D. (2014). Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239).

[16] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788.

[17] Ren, S., He, K., Girshick, R., and Sun, J. (2017). Faster r-cnn: Towards real-time object detection with region proposal networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6):1137–1149.

[18] Sekachev, B., Manovich, N., Zhiltsov, M., et al. (2020). Cvat: Computer vision annotation tool. *arXiv preprint arXiv:2009.13245*.

[19] Sun, X., Wu, J., Zhang, X., Zhang, Z., Zhang, C., Xue, T., Freeman, W., and Tenenbaum, J. (2017). Scenenet rgb-d: Can 5m synthetic images beat generic imagenet pre-training? In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2678–2687.

[20] Tan, M. and Le, Q. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 6105–6114.

[21] Wang, X., Luo, C., and Zhang, Y. (2019). Vision-based perception for autonomous indoor service robots: A survey. *IEEE Transactions on Automation Science and Engineering*, 16(4):1603–1619.